Video Super-Resolution with Temporal Consistency on REDS4 and a Frozen Vimeo Subset

Anonymous CVPR submission

Paper ID 3201

Abstract

Given a low-resolution video, we reconstruct a high-resolution video that recovers spatial detail and stays stable across frames. The shared input is the official 4× bicubic release, not a downsample recomputed by our own kernel. Part 1 compares bicubic and Lanczos upsampling, a frame-wise EDSR baseline, and a course temporal average. Part 2 runs EDVR and records the released TecoGAN checkpoint as a TensorFlow model that this Python environment cannot load. Part 3 treats Pyramid Flow as a conditioned generator only when the sampler can start from a noisy bicubic upsample, runs STAR from its released weights, and blends BasicVSR++ with STAR through a disagreement map. REDS4 is scored on RGB with no border crop. A 30-clip Vimeo subset, frozen from the official test list, is scored on the Y channel. The public repository link will be added here when the remote exists.

1. Introduction

Video super-resolution maps a low-resolution sequence to a high-resolution sequence. Doing this frame by frame can raise pixel scores while the restored detail flickers. The course task therefore asks for both spatial detail and temporal consistency.

We follow one comparison protocol. Every method sees the same official 4× bicubic low-resolution frames [10, 15]. A model trained on a different degradation is still run on that input, and the table names its training degradation. We do not pretrain Pyramid Flow or STAR. Where the assignment asks for an exploration that the released code does not support, we keep the attempt and write the failure instead of substituting an unrelated sample.

2. Related Work

Single-image networks learn a mapping from a low-resolution patch to a high-resolution patch. SRCNN introduced a convolutional mapping [4]. EDSR deepened residual blocks for bicubic super-resolution [8]. SRGAN traded distortion for sharper textures with an adversarial loss [7]. These models do not look at neighboring frames.

Video methods add alignment and fusion. EDVR aligns a short window with pyramid cascading deformable convolution and fuses it with temporal-spatial attention [12]. TDAN also aligns with deformable convolution over time [11]. BasicVSR argued that bidirectional propagation, feature alignment, and pixel-shuffle upsampling are the essential pieces [1]. BasicVSR++ adds second-order grid propagation and flow-guided deformable alignment [2].

Generative video models push the other side of the distortion–perception trade-off. TecoGAN uses a recurrent generator, a spatio-temporal discriminator, and a temporal loss, and it measures temporal change with tLP [3]. Flow matching learns a vector field from noise to data [9]. Pyramid Flow applies that idea in a pyramid for text-to-video and image-to-video generation [6]; it is not a published super-resolution model. STAR adapts a text-to-video backbone with ControlNet and a local insertion module for real-world video super-resolution [14]. LoRA is a low-rank update used as a course experiment here, not as STAR's training recipe [5].

Quality is not one number. SSIM compares structure [13]. LPIPS compares deep features [16]. REDS and Vimeo-90K supply the benchmark clips and the bicubic low-resolution track we evaluate [10, 15].

3. Method

3.1. Shared input

REDS4 is clips 000, 011, 015, and 020, held out from the REDS training split. Low-resolution frames come from train_sharp_bicubic/X4, the MATLAB imresize bicubic track used by EDVR and BasicVSR. Our bicubic and Lanczos kernels only upsample those frames. They are checked against OpenCV and are not used to rebuild the benchmark input.

The Vimeo benchmark is 30 seven-frame clips drawn once, without replacement, from BasicSR's 7824-clip official test list with numpy seed 0. The frozen list begins 00071/0454, 00080/0999, 00061/0846, 00053/0310, and 00075/0384. Tables call this set "Vimeo subset (30)". They do not call it the full Vimeo-90K-T.

3.2. Part 1

Bicubic upsampling uses OpenCV's cubic ($a = -0.75$), four taps, and replicated borders. Lanczos uses radius 4, eight taps, and renormalized weights. Both map output pixel i to source coordinate $(i+0.5)/\text{scale}-0.5$. EDSR is the official baseline with 16 residual blocks and 64 features, loaded from the DIV2K 4× checkpoint and not trained here [8].

The temporal average is a course binomial smoother, not a kernel from those papers. After bicubic upsampling, each frame is averaged with two neighbors on either side using weights $[1, 4, 6, 4, 1]/16$. Weights that fall off the clip are dropped and the rest are renormalized. Unsharp masking is an extra variant of that average, $y = x+1.0\,(x-G_{\sigma=1}(x))$, then clamped to the valid range. Sigma 1 and amount 1 are course defaults.

3.3. Part 2

EDVR-L uses the released bicubic checkpoint through BasicSR: 128 features, five input frames, deformable groups 8, TSA fusion, and reflection_circle padding [12]. On a 100-frame clip the window for the first frame is indices 4, 3, 0, 1, 2. The REDS blur-downsampling checkpoint is not this row. The Vimeo row, when the frames exist, uses the Vimeo-90K EDVR-L checkpoint.

The released TecoGAN archive is a TensorFlow 1 checkpoint [3]. env_cv is Python 3.10 and cannot install TensorFlow 1, so those weights are not loaded. A later PyTorch 4× checkpoint, if one runs, is reported separately and is not called the released model.

3.4. Part 3

Pyramid Flow's released model generates video from text, and its image-to-video path animates a clean first frame [6]. A result enters the metric table only when the official sampler can noise the whole bicubic-upscaled clip and integrate from a start near the middle of its own schedule. The table records that start index and the number of function evaluations. Official image-to-video and pure text-to-video samples do not. The pinned Pyramid Flow source (a012faa1) exposes only those two entry points, so this direction is recorded as a failure case in docs/results/pyramid_flow_attempt.json.

STAR's official row is inference with the released weights on the shared bicubic input. Its training degradation is real-world (OpenVid with a Real-ESRGAN-style pipeline), and the table says so [14]. A second row is a course LoRA experiment: rank 8, 200 steps, batch size 1, learning rate 10^{-4}, on 20 Vimeo-90K training septuplets that are not in the test subset [5]. Only injected LoRA weights are trained. The row is judged by whether tLPIPS drops. If the released code has no LoRA interface, the row is a failure case. Pinned STAR 69b8bc53 trains ControlNet and parameters named local, not LoRA, so that row is the failure recorded in docs/results/star_lora_attempt.json. The released light-degradation checkpoint is the official inference row. Its download is separate from the LoRA failure. The released U-Net calls xformers attention, and that package is pinned to an older PyTorch. This run keeps torch 2.11 and uses scaled dot-product attention for the same bias-free call.

The hybrid does not train a router. BasicVSR++ supplies the fidelity frame when a 4× bicubic checkpoint matched to the benchmark is available [2]. The generative frame is official STAR inference. Face and text boxes come from OpenCV FaceDetectorYN (YuNet) and TextDetectionModel_EAST, run on the bicubic-upscaled frame, with no dilation. Those detectors are course tools. Pixels inside the boxes are taken from BasicVSR++. Elsewhere the blend weight is the per-frame min–max normalized mean absolute RGB difference between the fidelity frame f and the generative frame g,

$$w = \text{norm}\left(\frac{1}{3}\sum_{c=1}^{3}|f_c - g_c|\right), \tag{1}$$

and the restored frame is $(1-w)f + wg$. Certain pixels set $w = 0$. There is no Gaussian smoothing and no extra global weight. This disagreement map is not a published uncertainty estimator. If STAR does not run, the hybrid is a failure case.

4. Experiments

4.1. Clip geometry

Each REDS4 clip has 100 frames. The official low-resolution frames are 180×320, and the ground-truth frames are 720×1280, so the shared scale is exactly 4. Four clips give 400 frames. The Vimeo subset has 30 septuplets, 210 frames in total, each ground-truth frame 256×448 before the official $4\times$ downsample. Full-reference scores use every frame. There is no border crop and no held-out frame inside a clip.

4.2. Protocol

PSNR and SSIM [13] on REDS4 use RGB and no border crop, including every frame, matching EDVR and BasicVSR. On the Vimeo subset they use the Y channel and no border crop. LPIPS uses AlexNet on RGB [16]. tLPIPS is TecoGAN's tLP [3]. With d the AlexNet LPIPS of two frames,

$$\text{tLPIPS} = \frac{1}{T-1}\sum_{t=2}^{T}\left|d(\hat{x}_t, \hat{x}_{t-1}) - d(x_t, x_{t-1})\right|. \quad (2)$$

We report that value, not the $\times 100$ display scale used in the paper's tables. On the Vimeo subset, PSNR and SSIM use the BT.601 luma of an RGB frame scaled to $[0, 1]$,

$$Y = 65.481R + 128.553G + 24.966B + 16. \quad (3)$$

FID pools the restored frames against the ground-truth frames of that set. The report states the frame count, so the number is not compared with a full-dataset paper FID. Scores use 8-bit quantized frames. Full-reference scores require ground truth. A wild video, which we do not have yet, will be judged visually.

4.3. What has been run

Table 1 records the state of each row. Every REDS4 method that ran has 100 frames in each of clips 000, 011, 015, and 020, and Table 2 gives the 400-frame scores. The MIT Vimeo host returns HTTP 403, so the 30-clip subset is not scored. The 7-frame Vimeo EDVR-L checkpoint is on disk and is not loaded into the 5-frame REDS model.

BasicVSR++ uses the released REDS4 bicubic checkpoint with 64 mid channels and 7 residual blocks [2]. The four clips are restored from train_sharp_bicubic/X4. YuNet and the EAST text graph are both on disk. An empty detector mask does not by itself fail the blend.

Pyramid Flow, the released TecoGAN weights, and the STAR LoRA row stay failure cases. STAR and the non-official PyTorch TecoGAN are scored on the shared bicubic input and labeled as a mismatched degradation and a substitute checkpoint. The released TecoGAN checkpoint has no score.

Table 1. REDS4 rows on the shared official $4\times$ bicubic input. A failure is a missing interface, not a skipped direction.

Row	State
Bicubic, Lanczos	Scored
EDSR baseline, DIV2K $4\times$	Scored, not trained
Temporal average, unsharp	Scored, course kernels
EDVR-L REDS bicubic	Scored
TecoGAN released	Failure: TensorFlow 1 checkpoint
TecoGAN PyTorch substitute	Scored, not the release
BasicVSR++ REDS4 bicubic	Scored
Pyramid Flow midpoint	Failure: no such sampler entry
STAR light degradation	Four clips, one frame at a time
STAR course LoRA	Failure: no LoRA parameters
Disagreement-map hybrid	Blended and scored



Figure 1. Clip 000, frame 50. Top: bicubic, EDVR-L, BasicVSR++, and official STAR. Bottom: the 160×160 patch with the strongest bicubic detail. In that patch the car edges are softer after bicubic upsampling than after EDVR-L or BasicVSR++. STAR was trained on real-world degradation, so this bicubic input is outside that training degradation.

Table 2 is the REDS4 score sheet. Scores use 8-bit quantized frames. The non-official TecoGAN row is listed so it cannot be confused with the released checkpoint.

Figure 1 is clip 000 at frame 50, a roadside scene with parked cars, a painted curb, and foliage. The bottom row is one 160×160 patch, chosen as the bicubic crop with the largest Laplacian variance, so the comparison is on detail rather than on a flat road. Bicubic upsampling leaves the car boundary soft. EDVR-L and BasicVSR++, both trained on REDS bicubic $4\times$, draw that boundary more sharply. STAR is in the same figure. Its car is sharper than bicubic and does not match the other restorations, which is consistent with a real-world degradation model run on bicubic input. The

Table 2. REDS4 full-reference scores. PSNR, SSIM, LPIPS, and tLPIPS are averaged over clips 000, 011, 015, and 020. PSNR and SSIM are RGB with no border crop. tLPIPS is not scaled by 100. FID is one Fréchet distance on all 400 frames. Blank cells are not zeros.

Method	PSNR	SSIM	LPIPS	tLPIPS	FID
Bicubic	26.24	0.7319	0.4577	0.0221	42.79
Lanczos	26.37	0.7365	0.4600	0.0229	36.74
EDSR	28.38	0.8077	0.2724	0.0042	16.36
Temporal average	22.07	0.6162	0.5746	0.1234	121.60
Unsharp	22.15	0.6215	0.5534	0.1189	119.21
EDVR-L	31.09	0.8800	0.1778	0.0095	8.16
BasicVSR++	32.39	0.9069	0.1315	0.0090	5.49
TecoGAN released			not loaded		
TecoGAN PyTorch, not released	27.62	0.7965	0.1646	0.0058	13.10
STAR light degradation	19.28	0.4602	0.4814	0.2614	95.92
STAR LoRA			no interface		
Pyramid Flow			no midpoint sampler		
Hybrid	27.26	0.8276	0.1848	0.0672	19.72



Figure 2. The same 160 × 160 patch under the five Part 1 restorations. Lanczos is a sharper interpolator than bicubic. The temporal average softens the car edge. Unsharp masking puts some of that contrast back. EDSR is the official DIV2K baseline and is not trained on REDS.

numbers for this comparison are in Table 2.

Figure 2 uses the same coordinates as Figure 1. Lanczos changes the interpolator only. The temporal average mixes five bicubic frames, so high-frequency edges move toward their neighbors and look softer. Unsharp masking is applied to that average, not to the bicubic frame, and it restores local contrast without looking at a learned prior. EDSR is a single-frame residual network trained on DIV2K. It does not see the neighboring REDS frames. These differences are why the later tables keep the five Part 1 rows separate from EDVR-L and BasicVSR++.

4.4. Inference settings

EDVR-L reads RGB scaled to $[0, 1]$ and emits RGB scaled back to $[0, 255]$ before quantization. The REDS checkpoint uses five frames. On a 100-frame clip, the window for frame 0 is indices 4, 3, 0, 1, 2 under reflection_circle padding. The Vimeo checkpoint uses seven frames, the whole septuplet, and is a different weight file from the REDS model. BasicVSR++ runs the low-resolution clip as one recurrent sequence, with 64 mid channels and 7 residual blocks. The released REDS4 checkpoint already contains SpyNet. Official STAR uses the light-degradation I2VGen-XL weights, the fast solver, and 15 function evaluations. Chunks of 2 or more frames at 720×1280 request about 31 GB and do not fit in 24 GB, so this run uses one frame at a time. Clip 000 is restored: 100 frames at 720×1280. REDS has no caption, so the text condition is a generic description of a real scene. That model was trained on real-world degradation, not on bicubic REDS, and the score table will say so. The hybrid detects faces with YuNet and text with EAST on the bicubic-upscaled low-resolution frame. Boxes are not dilated. Pixels inside a box are copied from BasicVSR++. Pixels outside a box use the disagreement map. If a detector returns no boxes, the blend still runs. On clip 000 the two detectors together mark 8640 pixels, and only 3 of the 100 frames contain any box. On clip 011 they mark 124291 pixels across 11 frames. On clip 015 they mark 16384 pixels across 2 frames. On clip 020 they mark 293036 pixels across 34 frames. All four REDS4 certain masks are cached. The course temporal average is applied after bicubic upsampling. Its weights are $[1, 4, 6, 4, 1]/16$. At the first frame, the two weights that would read before the clip are dropped and the remaining weights are renormalized. The edge is not filled by copying the first frame. Unsharp masking then sets $y = x + (x - G_{\sigma=1}(x))$ and clamps the result to $[0, 255]$. Sigma 1 and amount 1 are course defaults, not a published video kernel.

4.5. Recorded failures

Three rows are failures because the released interface does not implement the course request. The TecoGAN archive is 11288778 bytes and contains TecoGAN.data-00000-of-00001, TecoGAN.index, and TecoGAN.meta. Those are TensorFlow 1 checkpoint shards. No PyTorch state dict is in the archive, and the course environment cannot install TensorFlow 1. Pyramid Flow at a012faa1 has generate, which samples from noise, and generate_i2v, which keeps one clean image latent as history. It has no call that noises a whole bicubic-upscaled clip and integrates from the sampler midpoint, so that row stays out of the metric table. STAR at 69b8bc53 optimizes VideoControlNet and parameters whose names contain local. The optimizer list has no LoRA weights, so rank 8, 200 steps, batch size 1, and learning rate 10^{-4} were not run. A separate PyTorch generator trained on Vimeo bicubic data has now restored the four REDS4 clips. It is not the released TecoGAN checkpoint.

4.6. Reproducibility

The code, the frozen Vimeo list, and this draft live in one git repository. Part 1 is scripts/run_part1.py. EDVR-L on REDS4 is scripts/run_edvr.py with the 5-frame bicubic checkpoint. The Vimeo row is scripts/run_edvr_vimeo.py with the 7-frame checkpoint, and it refuses to

run without the official frames. BasicVSR++ is scripts/run_basicvsrpp.py. The non-official TecoGAN generator is scripts/run_tecogan_not_official.py. Official STAR is scripts/run_star_official.py, and the hybrid is scripts/run_hybrid.py. After train_sharp is extracted, scripts/score_ready.py fills RGB PSNR, SSIM, AlexNet LPIPS, tLPIPS, and small-sample FID for every method that has saved frames. Pinned sources are BasicSR 8d56e3a, Pyramid Flow a012faa1, and STAR 69b8bc53. The machine has eight 24 GB GPUs. One GPU is used per restoration so the runs do not share memory.

5. Discussion

The shared input is the official MATLAB bicubic release. Rebuilding the low-resolution frames with our own kernel would make every later number depend on that kernel, including the OpenCV match in Part 1. Our bicubic and Lanczos implementations are checked against OpenCV on float images, and they are used only to upsample. They are not used to synthesize a new benchmark.

REDS4 is four training clips held out of the fit, not the official REDS validation split. The validation split is a different set of clips. Reporting REDS4 as "REDS val" would name the wrong videos. The Vimeo table, when the official package can be downloaded, will say "Vimeo subset (30)" and will list 210 frames. It will not be labeled as the full Vimeo-90K test set of 7824 septuplets.

tLPIPS is the absolute gap between two LPIPS values: the restored pair and the ground-truth pair. It is not the LPIPS of a frame-difference image. A method can raise PSNR while increasing tLPIPS if the extra detail flickers. That is why the temporal average and the unsharp variant stay in the table even though they are course baselines. The temporal average should lower flicker and also lower sharpness. Unsharp masking should put contrast back and can put flicker back with it. On these 400 frames the temporal average scores 22.07 dB and tLPIPS 0.123, and the unsharp variant scores 22.15 dB and tLPIPS 0.119, against bicubic at 26.24 dB and tLPIPS 0.022.

The disagreement map is not an uncertainty estimate from EDVR, BasicVSR++, or STAR. It is the normalized absolute difference of two restorations on one frame. Certain pixels are an external prior: YuNet faces and EAST text on the bicubic-upscaled frame. If those detectors find nothing, every pixel uses the disagreement weight, and the hybrid is still the hybrid. A missing STAR restoration is the failure case. BasicVSR++ alone is not reported as the blend.

6. Conclusion

The pipeline separates official bicubic evaluation from course choices: the temporal-average kernel, the unsharp defaults, the Pyramid Flow midpoint, the STAR LoRA budget, and the disagreement map. Limitations follow from that split. Generative models trained on real-world degradations are scored on bicubic inputs, so a lower PSNR is not a pure architecture comparison. The Vimeo numbers, once the official test package is reachable, will describe 30 frozen clips, not the full test set. TecoGAN's released checkpoint cannot run in the course environment, and the PyTorch substitute is labeled as a different model. On the 400 REDS4 frames, BasicVSR++ reaches 32.39 dB and EDVR-L reaches 31.09 dB. Official STAR, run one frame at a time, reaches 19.28 dB with tLPIPS 0.261. The hybrid, which keeps detector boxes from BasicVSR++ and blends the remaining pixels by disagreement, reaches 27.26 dB. The Vimeo subset stays unscored while the MIT package returns HTTP 403.

References

[1] Kelvin C.K. Chan, Xintao Wang, Ke Yu, Chao Dong, and Chen Change Loy. BasicVSR: The search for essential components in video super-resolution and beyond. In CVPR, 2021. 1

[2] Kelvin C.K. Chan, Shangchen Zhou, Xiangyu Xu, and Chen Change Loy. BasicVSR++: Improving video super-resolution with enhanced propagation and alignment. In CVPR, 2022. 1, 2, 3

[3] Mengyu Chu, You Xie, Jonas Mayer, Laura Leal-Taixé, and Nils Thuerey. Temporally coherent GANs for video super-resolution. ACM Transactions on Graphics, 2020. 1, 2, 3

[4] Chao Dong, Chen Change Loy, Kaiming He, and Xiaoou Tang. Image super-resolution using deep convolutional networks. In IEEE Transactions on Pattern Analysis and Machine Intelligence, 2015. 1

[5] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In ICLR, 2022. 1, 2

[6] Yang Jin et al. Pyramidal flow matching for efficient video generative modeling. In ICLR, 2025. 1, 2

[7] Christian Ledig, Lucas Theis, Ferenc Huszár, Jose Caballero, Andrew Cunningham, Alejandro Acosta, Andrew Aitken, Alykhan Tejani, Johannes Totz, Zehan Wang, and Wenzhe Shi. Photo-realistic single image super-resolution using a generative adversarial network. In CVPR, 2017. 1

[8] Bee Lim, Sanghyun Son, Heewon Kim, Seungjun Nah, and Kyoung Mu Lee. Enhanced deep residual networks for single image super-resolution. In CVPR Workshops, 2017. 1, 2

[9] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In ICLR, 2023. 1

[10] Seungjun Nah, Sungyong Baik, Seokil Hong, Gyeongsik Moon, Sanghyun Son, Radu Timofte, and Kyoung Mu Lee. NTIRE 2019 challenge on video deblurring and super-resolution: Dataset and study. In CVPR Workshops, 2019. 1

[11] Yapeng Tian, Yulun Zhang, Yun Fu, and Chenliang Xu. TDAN: Temporally-deformable alignment network for video super-resolution. In CVPR, 2020. 1

[12] Xintao Wang, Kelvin C.K. Chan, Ke Yu, Chao Dong, and Chen Change Loy. EDVR: Video restoration with enhanced deformable convolutional networks. In CVPR Workshops, 2019. 1, 2

[13] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. IEEE Transactions on Image Processing, 2004. 1, 3

[14] Rui Xie et al. STAR: Spatial-temporal augmentation with text-to-video models for real-world video super-resolution. In ICCV, 2025. 1, 2

[15] Tianfan Xue, Baian Chen, Jiajun Wu, Donglai Wei, and William T. Freeman. Video enhancement with task-oriented flow. International Journal of Computer Vision, 2019. 1

[16] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In CVPR, 2018. 1, 3